



## ***Fase 3 do Projeto de Computação Gráfica***

*3º ano Licenciatura Engenharia Informática*

*Computação Gráfica*

*Universidade do Minho*

*2025/2026*

Grupo 11

Diogo Alves Ferreira (A106904)

Hugo Araújo Cunha (A106808)

José Miguel da Silva Santos (A72443)

Mariana Vivas Rodrigues (A106898)

# Índice

|                                                            |           |
|------------------------------------------------------------|-----------|
| <b>Índice.....</b>                                         | <b>2</b>  |
| <b>1. Introdução.....</b>                                  | <b>3</b>  |
| <b>2. Funcionalidades implementadas.....</b>               | <b>3</b>  |
| 2.1 Transformações hierárquicas e estrutura em grupos..... | 3         |
| 2.2 Rotações temporizadas.....                             | 4         |
| 2.2.1 Modelo implementado.....                             | 4         |
| 2.2.2 Aplicação e vantagens da abordagem.....              | 5         |
| 2.3 Translações com curvas de Catmull-Rom.....             | 5         |
| 2.3.1 Curvas Catmull-Rom.....                              | 6         |
| 2.3.2 Curvas fechadas.....                                 | 6         |
| 2.3.3 Aplicação na cena.....                               | 7         |
| 2.3.4 Alinhamento com a trajetória.....                    | 7         |
| 2.4 Bezier Patches.....                                    | 7         |
| 2.4.1 Superfícies de Bezier.....                           | 8         |
| 2.4.2 Tesselação da superfície.....                        | 8         |
| 2.4.3 Implementação no generator.....                      | 9         |
| 2.4.4 Aplicação na cena.....                               | 10        |
| 2.5 Implementação com VBOs.....                            | 10        |
| 2.5.1 Integração com a engine.....                         | 11        |
| 2.5.2 Impacto no desempenho.....                           | 11        |
| 2.6 Sistema solar dinâmico.....                            | 12        |
| <b>3. Conclusão.....</b>                                   | <b>14</b> |

# 1. Introdução

Na terceira fase do projeto de Computação Gráfica pretendeu-se expandir o motor gráfico desenvolvido nas fases anteriores, introduzindo novas técnicas de modelação, animação e otimização da renderização.

Enquanto nas fases anteriores o foco incidiu na geração de primitivas geométricas e na aplicação de transformações estáticas para construção de cenas hierárquicas, nesta fase foram acrescentadas funcionalidades que permitem enriquecer dinamicamente essas cenas. Entre elas destacam-se a implementação de transformações dependentes do tempo, a movimentação de objetos ao longo de curvas paramétricas, a geração de modelos através de patches de Bézier e a utilização de Vertex Buffer Objects (VBOs), permitindo melhorar significativamente a eficiência da renderização.

Adicionalmente, foi aprofundado o suporte para estruturas hierárquicas definidas em XML, possibilitando a construção de cenas complexas compostas por grupos de objetos com transformações independentes mas cumulativas.

Como demonstração das funcionalidades implementadas, foi desenvolvida uma cena dinâmica que integra transformações hierárquicas, trajetórias animadas e modelos gerados procedimentalmente.

## 2. Funcionalidades implementadas

### 2.1 Transformações hierárquicas e estrutura em grupos

Para permitir a construção de cenas complexas compostas por objetos relacionados entre si, foi implementado um sistema hierárquico baseado em grupos (Group), onde cada grupo pode conter:

- transformações;
- modelos;
- subgrupos.

Cada grupo representa um nó numa árvore hierárquica, sendo a cena representada pelo grupo raiz. A travessia desta árvore é feita recursivamente. Antes de aplicar as

transformações de um grupo, o estado atual da matriz de modelação é guardado através da função: `glPushMatrix()`.

Depois são aplicadas, pela ordem definida no ficheiro XML, todas as transformações associadas ao grupo: translação, rotação e escala.

De seguida desenham-se os modelos do grupo e processam-se recursivamente os subgrupos. Por fim a matriz original é restaurada com a função: `glPopMatrix()`.

Este mecanismo permite herdar automaticamente transformações dos grupos pai. Assim, por exemplo, uma lua pode orbitar um planeta enquanto acompanha simultaneamente o movimento orbital desse planeta em torno do sol.

## 2.2 Rotações temporizadas

Nas fases anteriores, as rotações suportadas pela *engine* eram exclusivamente estáticas, sendo usadas para posicionar ou orientar modelos num dado instante. Sendo aplicadas uma única vez durante o desenho da cena e mantendo-se constantes ao longo da execução.

Embora suficiente para construir cenas estáticas, esta abordagem tornava impossível representar movimentos contínuos, como por exemplo: rotação de planetas sobre o próprio eixo; movimento orbital e animações cíclicas e periódicas.

Assim, surgiu a necessidade de evoluir o sistema para suportar rotações dinâmicas dependentes do tempo que permitem animar objetos com movimento contínuo e periódico. Foi assim introduzida uma nova forma de definir rotações, onde se especifica o tempo necessário para completar uma rotação de 360°.

### 2.2.1 Modelo implementado

Se:

- $T$  representa o período de uma rotação completa;
- $t$  o tempo decorrido;

então o ângulo de rotação em cada instante é dado por:

$$\theta(t) = \frac{t}{T} * 360^\circ$$

Na implementação, o valor *elapsed* corresponde ao tempo decorrido desde o início da execução, enquanto *time* corresponde ao período definido no XML para uma rotação completa. Assim, o ângulo é recalculado em cada frame em função do tempo global, garantindo uma rotação contínua e independente da frame rate.

Desta forma o ângulo deixa de ser constante e passa a variar continuamente ao longo da execução.

### 2.2.2 Aplicação e vantagens da abordagem

Estas rotações foram utilizadas para simular a rotação dos planetas sobre o próprio eixo e o movimento orbital em torno do sol. Assim, permitiu representar movimentos naturais e contínuos, impossíveis com rotações estáticas. Além disso, como a animação depende do tempo real e não do número de *frames* renderizadas, a velocidade angular mantém-se constante independentemente do desempenho do sistema, tornando a animação independente da frame rate.

Finalmente, esta funcionalidade integra-se naturalmente na estrutura hierárquica já existente, permitindo compor movimentos mais complexos. Um objeto pode agora:

- possuir uma rotação própria;
- simultaneamente orbitar outro objeto;
- herdar transformações do grupo pai.

Isto permitiu aumentar significativamente o realismo e a expressividade das cenas desenvolvidas.

## 2.3 Translações com curvas de Catmull-Rom

Nas fases anteriores, as translações disponíveis eram estáticas  $T(x,y,z)$ , consistindo apenas num deslocamento fixo definido pelas coordenadas: `<translate x="..." y="..." z="..."/>`. Este tipo de transformação permitia posicionar objetos no espaço, mas não suportava trajetórias ou movimento contínuo.

Contudo, para representar movimentos mais naturais como órbitas, trajetórias fechadas e percursos suaves, tornou-se necessário desenvolver o sistema de transformações para suportar translações animadas. Foi assim introduzido suporte para trajetórias definidas por curvas Catmull-Rom, onde é possível definir uma transformação variável no tempo:  $T(P(t))$ , em que,  $P(t)$  é um ponto calculado na curva.

### 2.3.1 Curvas Catmull-Rom

As curvas Catmull-Rom permitem interpolar suavemente pontos de controlo fornecidos pelo utilizador. Dado:  $P_0, P_1, P_2, P_3$  um ponto da curva é calculado por:  $P(t) = T M P$  com:  $T = [t^3 \ t^2 \ t \ 1]$  e  $M$  (matriz Catmull-Rom).

Tal como nas rotações animadas, o parâmetro da curva depende do tempo:

$$t = \frac{elapsed}{T}$$

onde *elapsed* = tempo decorrido e  $T$  = período total. Desta forma o objeto percorre a trajetória completa em tempo constante. Para além da posição  $P(t)$ , a implementação calcula também a derivada da curva, que corresponde ao vetor tangente no ponto atual. Este vetor é essencial quando a opção **align** está ativa, pois permite orientar o objeto segundo a direção instantânea do movimento.

### 2.3.2 Curvas fechadas

Uma característica importante da nossa implementação é o facto das trajetórias Catmull-Rom serem tratadas como curvas fechadas. Esta decisão foi tomada tendo em conta que as animações pretendidas para a cena eram movimentos orbitais onde o objeto deve percorrer indefinidamente a mesma trajetória. Ao contrário de uma curva aberta, onde o percurso termina no último ponto de controlo, na nossa implementação o último segmento liga-se novamente ao primeiro, fechando naturalmente a trajetória.

Por exemplo, se existirem quatro pontos:  $P_0, P_1, P_2, P_3$  ao chegar ao último segmento, em vez da trajetória terminar em  $P_3$ , os pontos usados passam a ser:  $(P_2, P_3, P_0, P_1)$  ligando naturalmente o final ao início. Depois de escolhidos os pontos (`getCatmullRomPoint...`) é calculada a posição instantânea do objeto (*pos*) e tangente à curva (*deriv*), o que permite percorrer a trajetória fechada e integrá-la com o **align**.

Na nossa implementação, esta ligação entre o último e o primeiro ponto é obtida através da seleção cíclica dos pontos de controlo usando o operador módulo. Assim, quando o índice ultrapassa o último ponto da lista, volta automaticamente ao início, permitindo que a curva seja percorrida continuamente.

### 2.3.3 Aplicação na cena

Na cena desenvolvida, esta abordagem foi utilizada para representar movimentos periódicos contínuos, nomeadamente órbitas em torno de um corpo central.

Cada órbita é definida por um conjunto de pontos de controlo e percorrida ciclicamente ao longo do tempo. Combinando curvas fechadas, rotações temporizadas e transformações hierárquicas foi possível obter movimentos compostos em que um objeto orbita continuamente, roda sobre o próprio eixo e herda transformações do grupo pai.

### 2.3.4 Alinhamento com a trajetória

Para além de deslocar o objeto ao longo da curva, foi implementada a possibilidade de o alinhar com a direção do movimento através do atributo *align*. Quando esta opção está ativa, a tangente da curva é usada para definir a orientação local do objeto.

A tangente calculada em cada ponto da trajetória define a direção principal do movimento. A partir desta direção são construídos os restantes eixos locais do objeto, formando uma matriz de orientação. Esta matriz é então aplicada antes do desenho do modelo, fazendo com que o objeto acompanhe a curvatura da trajetória.

Esta funcionalidade é especialmente útil para objetos como asteroides ou cometas, pois evita que o modelo apenas “deslize” sobre a curva sem alterar a sua orientação.

## 2.4 Bezier Patches

Nas fases anteriores, o *generator* suportava apenas primitivas geométricas definidas analiticamente (planos, caixas, esferas e cones) e eram definidas por fórmulas geométricas. Embora permitissem modelar várias estruturas básicas, revelavam-se insuficientes para representar superfícies mais orgânicas ou geometrias complexas.

Assim, surgiu a necessidade de suportar superfícies paramétricas, permitindo gerar modelos suaves e detalhados a partir de um conjunto reduzido de pontos de controlo. Assim, foi introduzido suporte para **patches de Bézier**, permitindo gerar modelos a partir de ficheiros .patch e representar superfícies curvas complexas (*teapots*, modelos compostos por *patches*,...).

### 2.4.1 Superfícies de Bezier

Cada *patch* bicúbico é definido por 16 pontos de controlo, organizados numa matriz 4×4. Para calcular um ponto da superfície, a nossa implementação recorre aos polinómios de Bernstein, avaliando a contribuição de cada ponto de controlo para os parâmetros  $u$  e  $v$ , com  $u, v \in [0, 1]$ .

A superfície é então descrita por: 
$$P(u, v) = \sum_{i=0}^3 \sum_{j=0}^3 B_i(u) B_j(v) P_{ij}$$

onde  $P_{ij}$  representa cada ponto de controlo do patch e  $B_i(t)$  são os polinómios de Bernstein de grau 3:

$$B_0(t) = (1 - t)^3$$

$$B_1(t) = 3t(1 - t)^2$$

$$B_2(t) = 3t^2(1 - t)$$

$$B_3(t) = t^3$$

Desta forma, ao variar os parâmetros  $u$  e  $v$  no intervalo  $[0, 1]$ , é possível percorrer toda a superfície do patch. Cada ponto gerado resulta de uma soma ponderada dos 16 pontos de controlo, sendo o peso de cada ponto determinado pelos polinómios de Bernstein.

### 2.4.2 Tesselação da superfície

Como a superfície é contínua, esta necessita de ser discretamente desenhada. Para isso utiliza-se um parâmetro de tesselação, que subdivide o domínio paramétrico em pequenas células.



Se a tesselação for  $t$ , o intervalo:  $[0,1]$

é dividido em:  $step = \frac{1}{t}$

Percorrendo a superfície com dois ciclos sobre  $u$  e  $v$ , cada célula paramétrica origina quatro pontos:

- $(u,v)$
- $(u + step, v)$
- $(u, v + step)$
- $(u + step, v + step)$

que são triangulados em 2 triângulos por célula.

Para uma tesselação de nível  $t$ , cada patch origina  $t \times t$  células paramétricas. Como cada célula é dividida em dois triângulos, cada patch gera  $2t^2$  triângulos. Assim, aumentar a tesselação melhora a suavidade da superfície, mas também aumenta o número de vértices a processar e desenhar. Se for menor, menor será o número de triângulos e o detalhe.

### 2.4.3 Implementação no generator

Foi introduzido suporte para uma nova *primitive*: generator patch ficheiro.patch  
*tessellation* modelo.3d

O ficheiro .patch contém o número de patches, os índices dos pontos de controlo e as coordenadas dos pontos.

Para cada *patch*:

1. lêem-se os 16 pontos de controlo;
2. avalia-se a superfície para cada subdivisão;
3. geram-se os triângulos correspondentes;
4. os vértices são escritos no ficheiro .3d.

A lógica encontra-se implementada recorrendo à avaliação paramétrica da superfície e geração procedural dos triângulos.

#### 2.4.4 Aplicação na cena

Esta funcionalidade foi utilizada em cenas de teste específicas para representar modelos mais complexos, permitindo validar a geração de superfícies suaves de ficheiros .patch.

A utilização de *patches* permitiu enriquecer visualmente a cena e aumentar significativamente a capacidade de modelação do sistema.

Além disso, esta funcionalidade complementa naturalmente as fases anteriores:

- primitivas básicas para geometrias simples;
- patches para superfícies complexas.

A utilização de patches de Bézier apresenta várias vantagens, permite a modelação de superfícies suaves, representação compacta através de poucos pontos de controlo, geração procedural de modelos complexos e controlo geométrico intuitivo.

### 2.5 Implementação com VBOs

Nas fases anteriores, os modelos eram desenhados através do envio direto dos vértices para o OpenGL em cada frame. Embora esta abordagem fosse suficiente para cenas simples, tornava-se computacionalmente menos eficiente com o aumento do número de objetos e vértices, uma vez que os mesmos dados eram repetidamente enviados da CPU para a GPU.

Nesta fase foi introduzida a utilização de *Vertex Buffer Objects* (VBOs), permitindo armazenar os vértices dos modelos diretamente em memória gráfica. Assim, após a leitura do ficheiro .3d, os dados são enviados uma única vez para a GPU e passam a ser reutilizados em todas as frames seguintes.

Desta forma, a renderização torna-se mais eficiente, reduzindo a transferência repetida de dados entre CPU e GPU. Esta alteração é especialmente importante em cenas mais complexas, como o sistema solar dinâmico, onde vários modelos são desenhados continuamente ao longo da execução.

A utilização de VBOs baseia-se na separação entre duas fases distintas:

- Fase de carregamento: Após a leitura dos ficheiros .3d, os vértices de cada modelo são transferidos para a memória da GPU e associados a um buffer.

Esta operação é realizada apenas uma vez, no momento de inicialização ou carregamento da cena.

- **Fase de renderização:** Durante o desenho da cena, em vez de reenviar os vértices, a *engine* reutiliza os dados já armazenados na GPU, referenciando diretamente os buffers associados a cada modelo.

Desta forma, evita-se a comunicação repetida entre CPU e GPU, tornando o processo de renderização significativamente mais eficiente.

### 2.5.1 Integração com a *engine*

A introdução de VBOs foi feita de forma transparente relativamente à arquitetura já existente.

A estrutura hierárquica baseada em grupos mantém-se inalterada, isto é, cada grupo aplica as suas transformações e os modelos continuam a ser desenhados no contexto dessas transformações. No entanto, agora os dados geométricos são obtidos a partir de *buffers* na GPU.

Isto permitiu melhorar o desempenho sem alterar a lógica de organização da cena.

### 2.5.2 Impacto no desempenho

A utilização de VBOs revelou-se particularmente importante nesta fase devido a:

- elevado número de vértices, resultante da tesselação de superfícies de Bézier;
- modelos complexos, como o *teapot*;
- animação contínua da cena, exigindo renderização em cada frame.

Sem VBOs, o envio repetido de grandes quantidades de dados tornaria a aplicação significativamente mais lenta.

Com esta abordagem, foi possível reduzir o overhead de comunicação entre CPU e GPU, melhorar a fluidez da animação e suportar cenas mais complexas sem perda significativa de desempenho.

Desta forma, a utilização de VBOs constitui uma evolução fundamental da *engine*, permitindo suportar cenas mais complexas de forma eficiente e garantindo melhor desempenho na renderização em tempo real. Além disso, é particularmente relevante para

os modelos gerados por patches de Bézier, uma vez que o número de triângulos cresce quadraticamente com o nível de tesselação. Sem VBOs, estes vértices teriam de ser reenviados à GPU em todas as frames, penalizando a animação da cena.

## 2.6 Sistema solar dinâmico

Como demonstração das funcionalidades desenvolvidas na terceira fase, foi construída uma cena representando um sistema solar dinâmico.

A cena encontra-se organizada hierarquicamente em grupos, o que permite representar naturalmente relações de dependência entre objetos. O Sol ocupa a posição central e os restantes corpos celestes encontram-se organizados em subgrupos próprios. Cada planeta é colocado na sua órbita através de uma rotação temporizada em torno do eixo Y, seguida de uma translação que define a distância ao Sol. Para além do movimento orbital, cada planeta possui também uma rotação sobre o próprio eixo, igualmente dependente do tempo.

Do ponto de vista geométrico, a maior parte dos corpos da cena foi construída a partir do mesmo modelo base, uma esfera centrada na origem. A distinção entre Sol, planetas, luas e estrelas é obtida através da aplicação de diferentes escalas e transformações de posicionamento.

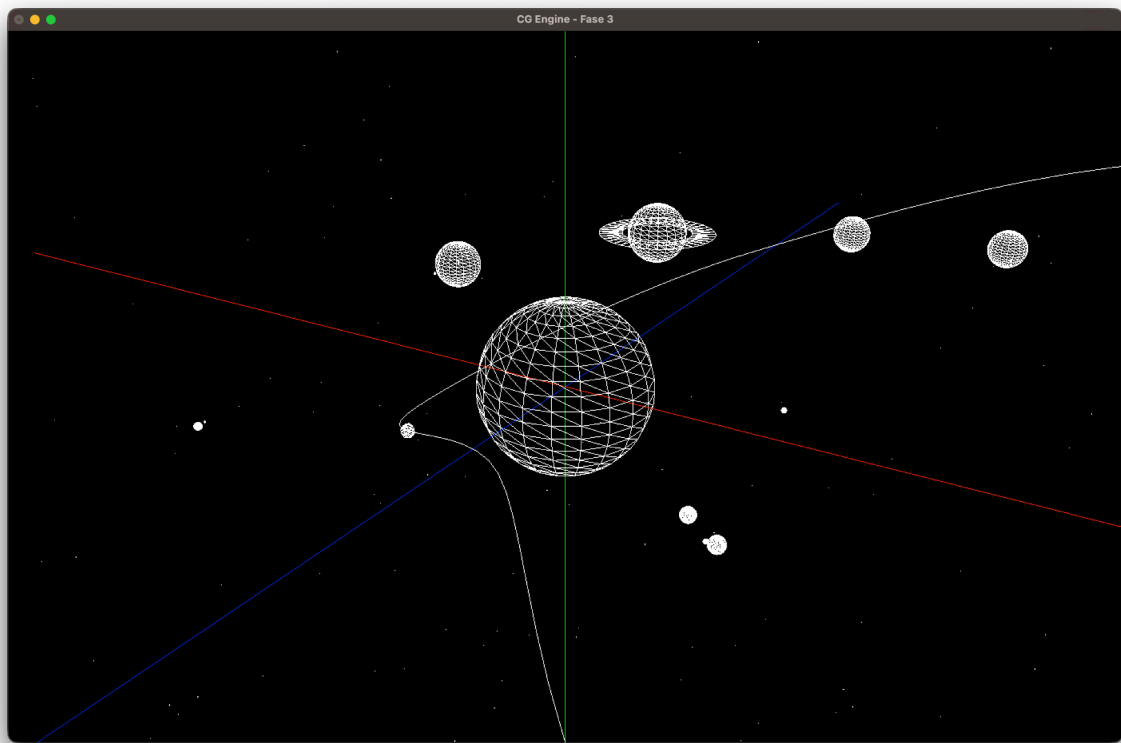
O Sol é representado por uma esfera de maior dimensão colocada na origem da cena, funcionando como corpo central do sistema. Em torno dele orbitam vários planetas, também modelados por esferas, cada um com escala própria e rotação sobre si mesmo. Entre estes encontram-se Mercúrio, Vénus, Terra, Marte, Júpiter, Saturno, Úrano e Neptuno.

Alguns planetas incluem ainda luas, organizadas como subgrupos adicionais. A Terra inclui a Lua; Marte inclui Fobos e Deimos; Júpiter inclui Europa e Ganimedes; Saturno inclui Titã; Urano inclui Titânia; e Neptuno inclui Tritão. Tal como os planetas, estas luas são representadas por esferas de menor dimensão, possuindo rotação própria e uma órbita local em torno do planeta correspondente. Graças à estrutura hierárquica, cada lua acompanha automaticamente o movimento do planeta pai em torno do Sol.

A cena inclui também elementos adicionais para enriquecer a demonstração. Saturno possui um anel, modelado por um objeto próprio, centrado no planeta e herdando a sua transformação global. Para além disso, foram distribuídas várias estrelas pela cena, representadas por pequenas esferas colocadas em posições fixas afastadas da região central.

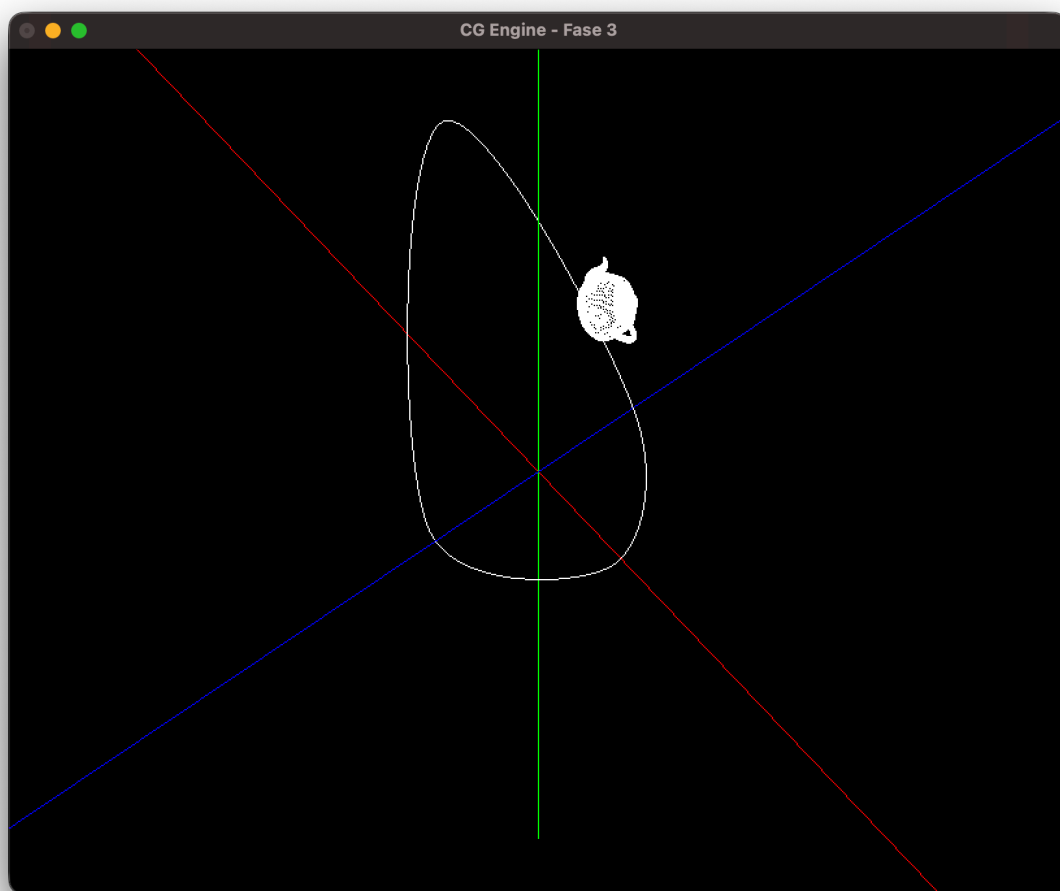
Finalmente, foi incluído um cometa animado, cujo movimento é definido por uma curva fechada de Catmull-Rom. Ao contrário dos planetas, este objeto não segue uma órbita circular simples, mas percorre uma trajetória suave determinada por um conjunto de pontos de controlo.

Do ponto de vista geométrico, este cometa foi construído a partir de um modelo gerado com Bezier patches, permitindo demonstrar a integração desta funcionalidade na cena principal e não apenas em testes isolados. Desta forma, a cena do sistema solar combina simultaneamente animação por curvas, organização hierárquica e geração procedural de superfícies, reunindo num único exemplo as principais funcionalidades desenvolvidas nesta fase.

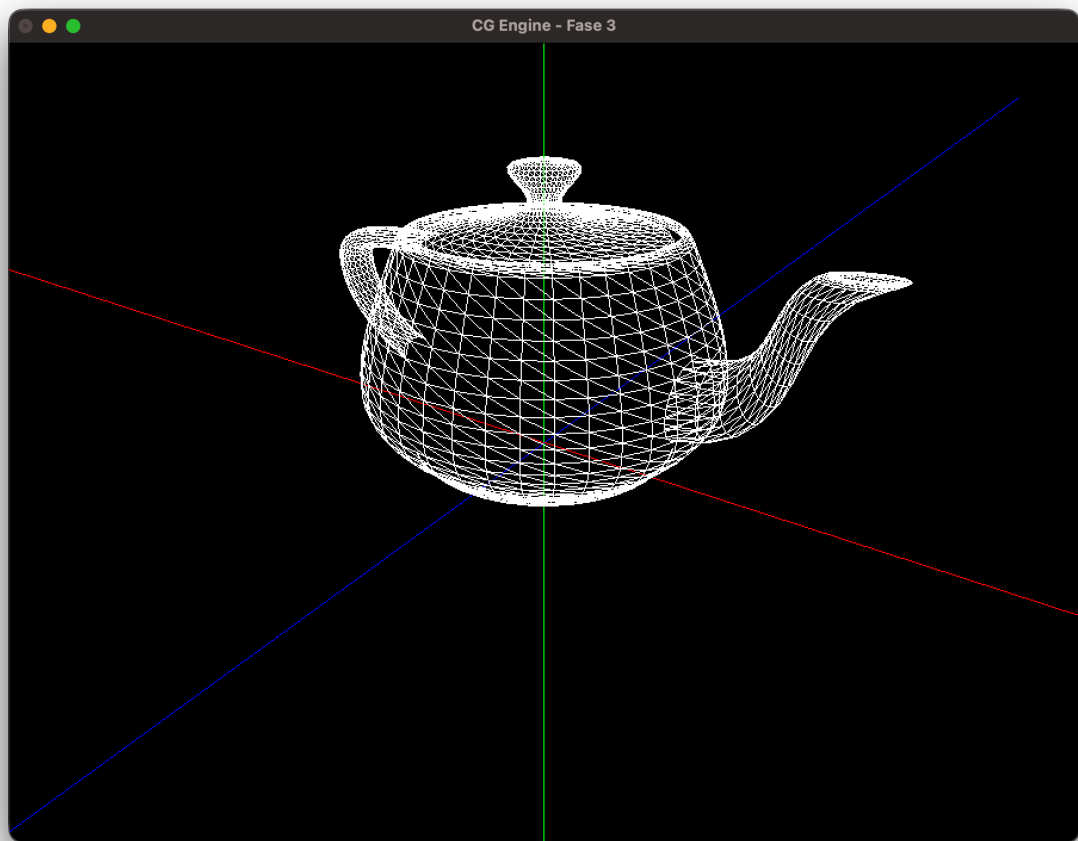


*Figura 1 - Sistema Solar desenvolvido*

### 3. Resultados e Validação



*Figura 2 - Teste 1 Fase 3*



*Figura 3 - Teste 2 Fase 3*

## 4. Conclusão

Ao longo desta terceira fase do projeto, foi possível evoluir significativamente o motor gráfico desenvolvido nas fases anteriores, introduzindo novas funcionalidades que permitiram enriquecer tanto a componente de modelação como de animação e desempenho.

Foram implementadas transformações dependentes do tempo, permitindo representar movimentos contínuos e periódicos, como rotações e órbitas, de forma independente da frame rate. A introdução de trajetórias baseadas em curvas de Catmull-Rom possibilitou a definição de percursos suaves e naturais, complementados pelo alinhamento automático dos objetos com a direção do movimento. Paralelamente, o suporte para *patches* de Bézier permitiu ultrapassar as limitações das primitivas geométricas básicas, possibilitando a geração de superfícies mais complexas e suaves a partir de um conjunto reduzido de pontos de controlo. A utilização de tesselação revelou-se fundamental para a discretização destas superfícies, evidenciando o compromisso entre detalhe geométrico e desempenho.

Do ponto de vista da eficiência, a introdução de VBOs constituiu uma melhoria essencial, permitindo reduzir significativamente o custo de comunicação entre CPU e GPU e garantindo uma renderização mais fluida, especialmente em cenários com elevado número de vértices.

A integração destas funcionalidades foi demonstrada através da construção de uma cena dinâmica representando um sistema solar, onde foram combinadas transformações hierárquicas, animações temporizadas e trajetórias paramétricas, evidenciando a flexibilidade e capacidade expressiva do sistema desenvolvido.

Em suma, esta fase permitiu não só expandir as capacidades técnicas da engine, mas também consolidar conceitos fundamentais de computação gráfica, nomeadamente no domínio da animação, modelação paramétrica e otimização da renderização.